

Product Research Agent

An autonomous, fully local AI pipeline that transforms a topic into a structured Product Requirements Document using Reddit, Hacker News, and Llama 3 — running entirely on your Mac at zero cost.

29 Nodes

3 AI Stages

2 Data Sources

100% Local

Author: Suparna
Deshmukh

Date: May 2026

Version: 1.0

Platform: macOS (Apple
Silicon)

1 | Project Overview

The Product Research Agent is a 29-node n8n workflow that automates the entire process of gathering, cleaning, classifying, and synthesising user feedback from Reddit and Hacker News into a polished product document. A user fills in a single web form — specifying a topic, platform, output format, and date range — and the agent handles everything else: keyword expansion, subreddit discovery, data fetching, AI classification, competitor detection, relevance filtering, aggregation, and final document generation.

The entire stack runs locally on a MacBook Air with Apple Silicon. No data leaves the machine. There are no subscription fees, no API keys, and no cloud dependencies. Llama 3 (4.9 GB) is served by Ollama and is invoked at multiple stages of the pipeline for tasks ranging from single-sentence classification to multi-page document authoring.

2 | Technology Stack

Component	Version	Role
n8n (self-hosted)	2.18.5	Workflow orchestration
Ollama	0.22.1	Local LLM runtime
Llama 3	latest (4.9 GB)	AI classification & generation
Node.js	25.9.0	n8n runtime
Python	3.14.0	Code nodes & file I/O

Reddit JSON API	Public	Community discussion data
HN Algolia API	Public	Tech discussion data
macOS	Apple Silicon	Host environment

3 | Pipeline Architecture

The workflow is organised into six logical phases that execute sequentially:

Phase 1 — Input & Keyword Expansion (Nodes 1–4)

The user submits the form. The topic is stored as variables and passed to Llama 3, which generates 10 semantically related search keywords. These are parsed into individual items for looping.

Phase 2 — Data Collection (Nodes 5–8)

The expanded keywords drive two parallel search paths. Reddit is queried to discover relevant subreddits (filtered to 10k+ subscribers), then each subreddit is searched for topic-specific posts. Simultaneously, Hacker News is queried via the Algolia API for each keyword.

Phase 3 — Data Cleaning & Merging (Nodes 9–11)

Raw API responses from both sources are stripped down to eight essential fields and merged into a single unified dataset.

Phase 4 — AI Analysis (Nodes 12–20)

Llama 3 classifies each post by theme and urgency, flags competitor mentions, and performs a relevance check. Posts that fail the relevance filter are discarded before aggregation.

Phase 5 — Aggregation & Document Generation (Nodes 21–25)

Posts are grouped by theme with counts and urgency averages. A format-specific prompt is constructed and sent to Llama 3, which authors the full output document. The response is cleaned and formatted.

Phase 6 — Output & Logging (Nodes 26–29)

The final document is saved as a timestamped markdown file. A structured log entry recording key run metadata is written to a separate logs directory.

4 | Node Directory

The table below documents every node in the workflow, its type, and its function.

#	Node Name	Type	Description
1	Form Trigger	Trigger	Entry point of the workflow. Presents the user with a web form capturing four inputs: Topic (free text), Platform (Reddit Only / HN Only / Both), Output Format (Full PRD / Pain Points Only / User Stories Only), and Date Range (Past Week / Past Month / Past 3 Months).
2	Set	Transform	Stores the four form values as clean, named variables — topic, platform, output_format, and date_range — that every downstream node can reference without re-parsing the raw form payload.
3	Expand Keywords	HTTP / AI	Sends the topic to Llama 3 via a local Ollama HTTP call and requests exactly 10 semantically related search keywords. These keywords broaden the research surface beyond the literal topic phrase.
4	Parse Keywords	Code	Converts Llama 3's raw text response into a clean JavaScript array and splits it into 10 individual items so n8n can loop over each keyword independently in subsequent nodes.
5	Search Reddit Posts	HTTP	Issues a Reddit JSON search request for each keyword, fetching the top 10 posts sorted by relevance over the chosen date range. A 2-second batch interval prevents rate limiting.
6	Extract Subreddits	Code	Reads the subreddit name and subscriber count from every returned post. Deduplicates the list and keeps only subreddits with more than 10,000 members, ensuring the agent focuses on active, high-signal communities.
7	Fetch Reddit Posts	HTTP	For every verified subreddit, performs a targeted search using the original topic string restricted to that subreddit. Returns up to 10 posts per subreddit, batched with a 2-second delay.
8	Fetch HN Stories	HTTP	Queries the Hacker News Algolia API for each expanded keyword, retrieving the top 5 stories per keyword. Hacker News provides a complementary technical and startup-oriented perspective alongside Reddit's community discussions.
9	Clean Reddit Data	Code	Strips every Reddit post down to eight essential fields: source, subreddit, title, body (capped at 1,000 characters), upvotes, comment count, URL, and date. Removes deleted or empty posts.
10	Clean HN Data	Code	Applies the same eight-field schema to Hacker News stories, using 'N/A' for the subreddit field and falling back to the HN item URL when no external link is present. Ensures both datasets are structurally identical.
11	Merge All Sources	Merge	Combines the Reddit stream and the HN stream into a single unified dataset using n8n's Append merge mode. All downstream nodes work from one list regardless of source.
12	Build Classification Prompt	Code	Prepares a structured JSON request body for each post, embedding the title and a 150-character body excerpt. The prompt instructs Llama 3 to classify the post into one of seven themes and assign a 1–5 urgency score.
13	Classify Themes	HTTP / AI	Sends each prepared prompt to Llama 3 and receives a JSON object containing theme, urgency score, and a one-sentence summary. Runs with a 2-minute timeout to accommodate local model response times.
14	Parse Classifications	Code	Extracts the structured classification from Llama 3's response and merges it back with the original post data from the Build Classification Prompt node, producing a fully enriched record per post.

#	Node Name	Type	Description
15	Build Competitor Prompt	Code	Combines each post's title and body into a single text block and constructs a competitor-detection prompt. Skips posts with fewer than 5 characters of combined content.
16	Flag Competitors	HTTP / AI	Asks Llama 3 whether each post mentions a competitor product or service by name. Returns a structured JSON flag with competitor_mentioned, competitor_name, and competitor_context.
17	Parse Competitor Flags	Code	Merges the competitor detection result back with the full post record from Build Competitor Prompt. Every post now carries three extra fields: competitor_mentioned, competitor_name, and competitor_context.
18	Build Relevance Prompts	Code	Constructs a short yes/no relevance-check prompt for every post, asking Llama 3 whether the post is specifically about the researched topic. Passes the original post fields alongside the request body.
19	Check Relevance	HTTP / AI	Sends each relevance prompt to Llama 3 and receives a {relevant: true/false} response. This AI-powered filter catches off-topic posts that keyword matching alone cannot reliably eliminate.
20	Apply Relevance Filter	Code	Reads the relevance verdict from Check Relevance and discards posts where Llama 3 returned false. A safety net retains the top 10 most-upvoted posts if the filter removes too many items.
21	Aggregate by Theme	Code	Groups all surviving posts by theme and computes, per theme: post count, average urgency score, top 5 summaries, and deduplicated competitor mentions. Themes are sorted by frequency descending.
22	Build PRD Prompt	Code	Constructs a comprehensive, format-specific prompt using the aggregated theme data. Switches between three prompt templates based on the user's Output Format selection: Full PRD, Pain Points Only, or User Stories Only. Includes instructions for competitive analysis and specific user stories.
23	Generate PRD	HTTP / AI	Sends the full structured prompt to Llama 3 with a 5-minute timeout. Llama 3 writes the complete output document — a full PRD, pain point analysis, or user story map — in markdown format.
24	Parse PRD Response	Code	Cleans the raw Llama 3 output by stripping markdown code fences, trimming whitespace, and ensuring the document starts at the first heading. Passes topic, format, date, and word count to downstream nodes.
25	Format Output	Code	Adds a professional header banner (topic, output type, generation date) and fixes common Llama 3 formatting issues: missing line breaks around headers, inconsistent bullet spacing, excessive blank lines, and malformed checkboxes.
26	Save Markdown File	Code	Converts the final formatted document into binary data and constructs a descriptive filename — e.g. whey-protein-powder-prd-2026-05-02.md — using the topic, output format suffix, and today's date.
27	Write PRD File	File I/O	Writes the binary markdown file to disk at /Users/suparna_deshmukh/projects/product_research_agent/outputs/. Creates the directory automatically if it does not exist.
28	History Log	Code	Builds a structured plain-text log entry recording the run date, topic, output format, themes found, word count, and output filename. Converts the entry to binary for writing.
29	Write History Log	File I/O	Saves the log entry as a uniquely named file — e.g. log-whey-protein-powder-2026-05-02.txt — in the logs/ directory, creating a permanent audit trail of every research run.

5 | Data Flow Summary

The diagram below illustrates how data transforms as it moves through the six pipeline phases:

Stage	Input	Output
Form	User interaction	4 variables: topic, platform, format, date_range
Keyword Expansion	1 topic string	10 search keywords (individual items)
Data Collection	10 keywords	Raw Reddit posts + HN stories (~100–300 items)
Cleaning & Merge	Raw API responses	Unified dataset: 8 fields per item
AI Analysis	Unified dataset	Each item + theme, urgency, summary, competitor flag
Relevance Filter	All classified items	Only topic-relevant items retained
Aggregation	Filtered items	One record per theme with counts and averages
PRD Generation	Theme aggregates + prompt	Full markdown document (1,500–3,000 words)
File Output	Markdown document	.md file + .txt log saved to disk

6 | Output Formats

The agent produces one of three document types depending on the user's selection in the form. All outputs are saved as timestamped markdown files and are immediately readable in any markdown editor or viewer.

Output Format	Sections Included	Typical Length
Full PRD	Executive Summary, Problem Statement, Research Methodology, Pain Points by Theme, Competitor Landscape, User Stories	2,500–3,500 words
Pain Points Only	Executive Summary, Pain Points by Theme, Top 5 Critical Pain Points, Summary of Key Findings	1,500–2,000 words
User Stories Only	Executive Summary, User Personas, User Stories by Theme (with Acceptance Criteria), Competitor Pain Point Analysis	1,500–2,000 words

7 | File Output Structure

Every workflow run produces two files saved under the project directory:

File Type	Location	Naming Convention	Example
PRD / Report (.md)	outputs/	{topic}-{format}-{date}.md	whey-protein-powder-prd-2026-05-02.md
Run Log (.txt)	logs/	log-{topic}-{date}.txt	log-whey-protein-powder-2026-05-02.txt

8 | Key Design Decisions

Local-first architecture

All computation — LLM inference, data storage, file I/O — runs on the local machine. This eliminates API costs, protects research confidentiality, and removes internet latency from the AI stages.

Reddit JSON API over official credentials

Reddit's official API now requires pre-approval that can take days or weeks. The public JSON endpoint (appending .json to any Reddit URL) provides identical data instantly, with no credentials required, making the agent deployable by anyone immediately.

AI-powered relevance filtering

Keyword matching alone cannot reliably exclude off-topic posts. A dedicated Llama 3 relevance check node asks the model whether each post is specifically about the researched topic, providing a semantic filter that works correctly for any topic without configuration.

Batch interval rate limiting

Reddit limits request frequency. All HTTP Request nodes that hit Reddit use n8n's built-in batching with a 2-second interval, ensuring the agent never triggers a 429 response regardless of dataset size.

Competitor data quality filtering

Raw competitor detection from LLMs can return Reddit usernames or informal references as competitor names. The Build PRD Prompt node filters these out before they reach the document, retaining only plausible company names.

Format-driven prompt switching

Rather than generating a generic document and trimming it, the agent builds a completely different prompt for each output format. This produces documents that are purpose-built — a Pain Points report reads differently from a User Stories document — rather than truncated versions of the same text.